

✓ Approach B: RoBERTa + contextual augmentation (Team 14 / paper replication)

This notebook implements **Approach B** from the midsem deck:

- **Label mapping** (Kaggle Mental Health Text → 3 severity tiers):
 - **Normal** → **No Symptoms** (class 0)
 - **Anxiety**, **Depression** → **Mild/Moderate** (class 1)
 - **Suicidal** → **Severe** (class 2)
- **Model**: **roberta-base** sequence classification.
- **Imbalance**: contextual augmentation for the **Severe** class using masked-language substitutions (Kobayashi-style contextual augmentation, as cited in the base paper).

Colab: Runtime → GPU recommended. Upload **mental_heath_unbanlanced.csv** or mount Google Drive and set **CSV_PATH** below.

```
# Install compatible dependencies (Colab-safe)
```

```
%pip install -q transformers datasets accelerate scikit-learn pandas tqdm sentencepiece
```

```
import json
import os
import random
import re
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from datasets import Dataset
from sklearn.metrics import accuracy_score, classification_report, f1_score, recall_score
from sklearn.model_selection import train_test_split
from tqdm.auto import tqdm
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    Trainer,
    TrainingArguments,
)

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# --- Paths (edit for Colab) ---
# Example after uploading to Colab:
CSV_PATH = "/content/mental_heath_unbanlanced.csv"
# Or Google Drive:
# from google.colab import drive
# drive.mount("/content/drive")
# CSV_PATH = "/content/drive/MyDrive/.../mental_heath_unbanlanced.csv"

CSV_PATH = os.environ.get("CSV_PATH", "mental_heath_unbanlanced.csv")
OUTPUT_DIR = Path(os.environ.get("OUTPUT_DIR", "approach_b_roberta_model"))
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_NAME = "roberta-base"
MAX_LENGTH = 128

# Augmentation: BERT contextual substitutes for Severe class (training split only).
# Full augmentation on all Severe rows can be slow; raise cap for stricter replication.
AUGMENTS_PER_SEVERE_SAMPLE = 1
MAX_SEVERE_SAMPLES_TO_AUGMENT = 4000 # set to None to augment every Severe training example

print("CUDA:", torch.cuda.is_available(), torch.cuda.get_device_name(0) if torch.cuda.is_available() else "")
```

```
CUDA: True Tesla T4
```

1. Load CSV and map labels (PPT specification)

```
df = pd.read_csv("/content/mental_heath_unbanlanced.csv")
assert "text" in df.columns and "status" in df.columns, df.columns.tolist()

RAW_TO_SEVERITY = {
    "Normal": "No Symptoms",
    "Anxiety": "Mild/Moderate",
    "Depression": "Mild/Moderate",
    "Suicidal": "Severe",
}

label2id = {"No Symptoms": 0, "Mild/Moderate": 1, "Severe": 2}
id2label = {v: k for k, v in label2id.items()}

unknown = set(df["status"].unique()) - set(RAW_TO_SEVERITY.keys())
if unknown:
    raise ValueError(f"Unexpected status values: {unknown}")

df["severity_label"] = df["status"].map(RAW_TO_SEVERITY)
df["label"] = df["severity_label"].map(label2id)
df[["status", "severity_label", "label"]].value_counts()
```

count			
status	severity_label	label	
Normal	No Symptoms	0	18391
Depression	Mild/Moderate	1	14506
Suicidal	Severe	2	11212
Anxiety	Mild/Moderate	1	5503

dtype: int64

2. Text cleaning (light normalization)

```
def clean_text(text: str) -> str:
    text = str(text).strip()
    text = re.sub(r"http\S+", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["text_clean"] = df["text"].map(clean_text)
df = df[df["text_clean"].str.len() > 0].reset_index(drop=True)
df["severity_label"].value_counts()
```

count	
severity_label	
Mild/Moderate	20009
No Symptoms	18391
Severe	11211

dtype: int64

3. Stratified train / validation / test split

```
import os
import pandas as pd

# Check if we already have the saved augmented data
if os.path.exists('train_augmented.csv'):
    print("Loading existing augmented datasets from CSV...")
    df_train_aug = pd.read_csv('train_augmented.csv')
    df_val = pd.read_csv('val.csv')
```

```

df_test = pd.read_csv('test.csv')

X_train_aug, y_train_aug = df_train_aug['text'].tolist(), df_train_aug['label'].tolist()
X_val, y_val = df_val['text'].tolist(), df_val['label'].tolist()
X_test, y_test = df_test['text'].tolist(), df_test['label'].tolist()

print(f"Loaded: Train({len(X_train_aug)}), Val({len(X_val)}), Test({len(X_test)})")
else:
    # Original splitting logic if CSVs don't exist
    X = df["text_clean"].tolist()
    y = df["label"].tolist()

    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.2, random_state=SEED, stratify=y
    )
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=SEED, stratify=y_temp
    )
    print("Train", len(X_train), "Val", len(X_val), "Test", len(X_test))

```

Train 39688 Val 4961 Test 4962

4. Contextual augmentation (Severe class, train only)

Uses contextual masked-language substitution, consistent with the paper's discussion of contextual augmentation / contextualized embeddings for minority classes.

```

if os.path.exists('train_augmented.csv'):
    print("Augmented data already exists. Skipping BERT-based augmentation loop.")
else:
    from transformers import pipeline
    import torch

    print("Initializing augmentation pipeline...")
    aug_device = 0 if torch.cuda.is_available() else -1
    mask_filler = pipeline("fill-mask", model="bert-base-uncased", device=aug_device)

    def augment_severe_texts(texts: list[str], labels: list[int]) -> tuple[list[str], list[int]]:
        severe_idx = [i for i, lab in enumerate(labels) if lab == label2id["Severe"]]
        if MAX_SEVERE_SAMPLES_TO_AUGMENT is not None and len(severe_idx) > MAX_SEVERE_SAMPLES_TO_AUGMENT:
            rng = random.Random(SEED)
            severe_idx = rng.sample(severe_idx, MAX_SEVERE_SAMPLES_TO_AUGMENT)

        new_texts = list(texts)
        new_labels = list(labels)
        severe_label = label2id["Severe"]

        for i in tqdm(severe_idx, desc="BERT-based aug (Severe)"):
            text = texts[i]
            words = text.split()
            if len(words) < 5: continue
            try:
                idx_to_mask = random.randint(0, len(words) - 1)
                original_word = words[idx_to_mask]
                words[idx_to_mask] = "[MASK]"
                masked_sent = " ".join(words)
                predictions = mask_filler(masked_sent)
                for pred in predictions:
                    replacement = pred['token_str'].strip()
                    if replacement.isalpha() and replacement.lower() != original_word.lower():
                        words[idx_to_mask] = replacement
                        break
                augmented_text = clean_text(" ".join(words))
                if augmented_text != text:
                    new_texts.append(augmented_text)
                    new_labels.append(severe_label)
            except Exception: continue
        return new_texts, new_labels

    X_train_aug, y_train_aug = augment_severe_texts(X_train, y_train)
    print("Train after Severe aug:", len(X_train_aug))

```

```

Initializing augmentation pipeline...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
config.json: 100% 570/570 [00:00<00:00, 18.1kB/s]

model.safetensors: 100% 440M/440M [00:04<00:00, 174MB/s]

Loading weights: 100% 202/202 [00:00<00:00, 395.74it/s, Materializing param=cls.predictions.transform.dense.weight]
BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+---
cls.seq_relationship.weight | UNEXPECTED | |
bert.pooler.dense.bias | UNEXPECTED | |
bert.pooler.dense.weight | UNEXPECTED | |
cls.seq_relationship.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.20kB/s]

vocab.txt: 232k/? [00:00<00:00, 850kB/s]

tokenizer.json: 466k/? [00:00<00:00, 1.09MB/s]

BERT-based aug (Severe): 100% 4000/4000 [01:31<00:00, 70.56it/s]
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset
Token indices sequence length is longer than the specified maximum sequence length for this model (1845 > 512). Running
Train after Severe aug: 43583

```

5. Tokenize and Hugging Face Dataset

```

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def encode_batch(batch):
    enc = tokenizer(
        batch["text"],
        truncation=True,
        max_length=MAX_LENGTH,
        padding="max_length",
    )
    enc["labels"] = batch["label"]
    return enc

train_ds = Dataset.from_dict({"text": X_train_aug, "label": y_train_aug})
val_ds = Dataset.from_dict({"text": X_val, "label": y_val})
test_ds = Dataset.from_dict({"text": X_test, "label": y_test})

train_ds = train_ds.map(encode_batch, batched=True, remove_columns=["text"])
val_ds = val_ds.map(encode_batch, batched=True, remove_columns=["text"])
test_ds = test_ds.map(encode_batch, batched=True, remove_columns=["text"])

train_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
val_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
test_ds.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

```

```

config.json: 100%                                481/481 [00:00<00:00, 48.1kB/s]

tokenizer_config.json: 100%                        25.0/25.0 [00:00<00:00, 2.73kB/s]

vocab.json:      899k/? [00:00<00:00, 3.96MB/s]
merges.txt:      456k/? [00:00<00:00, 950kB/s]
tokenizer.json:   1.36M/? [00:00<00:00, 1.78MB/s]

Map: 100%                                43583/43583 [00:13<00:00, 3373.73 examples/s]
Map: 100%                                4961/4961 [00:01<00:00, 3751.30 examples/s]
Map: 100%                                4962/4962 [00:01<00:00, 2847.25 examples/s]

```

6. Fine-tune RoBERTa

```

num_labels = len(label2id)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=num_labels,
    id2label=id2label,
    label2id=label2id,
)

# Move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print(f"Model is training on: {device}")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    macro_f1 = f1_score(labels, preds, average="macro")
    recalls = recall_score(labels, preds, average=None, labels=[0, 1, 2], zero_division=0)
    severe_recall = float(recalls[label2id["Severe"]])
    return {"accuracy": acc, "macro_f1": macro_f1, "recall_severe": severe_recall}

# --- Use Entire Dataset ---
train_ds_fast = train_ds

args = TrainingArguments(
    output_dir=str(OUTPUT_DIR / "checkpoints"),
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=1,
    weight_decay=0.01,
    eval_strategy="steps",
    eval_steps=100,
    save_strategy="no",
    load_best_model_at_end=False,
    seed=SEED,
    report_to="none",
    fp16=torch.cuda.is_available(),
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds_fast,
    eval_dataset=val_ds.select(range(min(500, len(val_ds)))),
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()

```

Loading weights: 100% 197/197 [00:00<00:00, 587.20it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]

RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
roberta.embeddings.position_ids	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your down

Model is training on: cuda

[2724/2724 06:04, Epoch 1/1]

Step	Training Loss	Validation Loss	Accuracy	Macro F1	Recall Severe
100	No log	0.499983	0.796000	0.789971	0.829060
200	No log	0.452396	0.822000	0.816970	0.880342
300	No log	0.463482	0.820000	0.790148	0.512821
400	No log	0.442575	0.820000	0.815786	0.931624
500	0.538577	0.391863	0.850000	0.843901	0.871795
600	0.538577	0.359087	0.856000	0.848607	0.820513
700	0.538577	0.363320	0.866000	0.861075	0.854701
800	0.538577	0.386188	0.856000	0.850473	0.846154
900	0.538577	0.329257	0.868000	0.859604	0.803419
1000	0.417961	0.345893	0.854000	0.849459	0.897436
1100	0.417961	0.347127	0.862000	0.856631	0.854701
1200	0.417961	0.321947	0.856000	0.851149	0.897436
1300	0.417961	0.313281	0.868000	0.861996	0.863248
1400	0.417961	0.305523	0.878000	0.872558	0.871795
1500	0.383951	0.291194	0.872000	0.865806	0.863248
1600	0.383951	0.284514	0.876000	0.866645	0.786325
1700	0.383951	0.277114	0.890000	0.882029	0.829060
1800	0.383951	0.293582	0.872000	0.866365	0.888889
1900	0.383951	0.273164	0.894000	0.887192	0.846154
2000	0.343121	0.274882	0.886000	0.880214	0.880342
2100	0.343121	0.278480	0.886000	0.880411	0.888889
2200	0.343121	0.265761	0.898000	0.892063	0.871795
2300	0.343121	0.275155	0.886000	0.880569	0.905983
2400	0.343121	0.263883	0.888000	0.881322	0.871795
2500	0.325707	0.263358	0.892000	0.885234	0.854701
2600	0.325707	0.257749	0.892000	0.885072	0.854701
2700	0.325707	0.261443	0.890000	0.883784	0.880342

TrainOutput(global_step=2724, training_loss=0.39769872869864087, metrics={'train_runtime': 365.884, 'train_samples_per_second': 119.117, 'train_steps_per_second': 7.445, 'total_flos': 2866818021209856.0, 'train_loss': 0.39769872869864087, 'epoch': 1.0})

7. Test set metrics (macro-F1 + per-class, esp. Severe recall)

```
preds_output = trainer.predict(test_ds)
y_true = preds_output.label_ids
```

```

y_pred = np.argmax(preds_output.predictions, axis=-1)

target_names = [id2label[i] for i in range(num_labels)]
print(classification_report(y_true, y_pred, target_names=target_names, digits=4))
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Macro-F1:", f1_score(y_true, y_pred, average="macro"))

```

	precision	recall	f1-score	support
No Symptoms	0.9741	0.9391	0.9563	1840
Mild/Moderate	0.8964	0.8041	0.8477	2001
Severe	0.7071	0.8787	0.7836	1121
accuracy			0.8710	4962
macro avg	0.8592	0.8740	0.8625	4962
weighted avg	0.8824	0.8710	0.8735	4962

Accuracy: 0.8710197501007658
Macro-F1: 0.8625424534439108

8. Save model + `label_map.json` for the standalone chatbot

```

trainer.save_model(str(OUTPUT_DIR))
tokenizer.save_pretrained(str(OUTPUT_DIR))

label_map = {
    "id2label": {str(k): v for k, v in id2label.items()},
    "label2id": label2id,
    "raw_kaggle_mapping": RAW_TO_SEVERITY,
}
with open(OUTPUT_DIR / "label_map.json", "w", encoding="utf-8") as f:
    json.dump(label_map, f, indent=2, ensure_ascii=False)

print("Saved to:", OUTPUT_DIR.resolve())

```

Writing model shards: 100%

1/1 [00:03<00:00, 3.50s/it]